

PROCEDIMIENTO Y DOCUMENTACIÓN CONSOLIDADA DE PRUEBAS

Servicios backend — RentaMóvil

RENTAMOVIL

**Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez
Vargas**

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

Proyecto	RentaMóvil — cinco módulos backend Java
Fecha de elaboración	9 de octubre de 2026
Versión	1.0
Elaborado por	Equipo de desarrollo
Estado	Línea base de pruebas ejecutada; brechas identificadas

1. CONTROL DE CAMBIOS

Versión	Fecha	Descripción	Responsable
1.0	2026-10-09	Consolidación del procedimiento, inventario y resultados de pruebas de los cinco módulos backend Java.	Equipo de desarrollo

2. OBJETIVO

Establecer un procedimiento común, repetible y verificable para diseñar, implementar, ejecutar y registrar las pruebas de los servicios backend de RentaMóvil. El documento consolida el inventario actual, los resultados por módulo, las limitaciones observadas y las acciones recomendadas.

Se documentan capacidades existentes y verificadas. Las pruebas sugeridas se presentan como mejoras pendientes, no como cobertura ya implementada.

3. ALCANCE

Este documento cubre los cinco módulos backend Java:

- `rtm-iam`: identidad y acceso.
- `rtm-fleet-maintenance`: flota y mantenimiento.
- `rtm-booking-reservation`: reservas y rentas.
- `rtm-payment-billing`: pagos y facturación.
- `rtm-api-gateway`: entrada y enrutamiento de solicitudes.

Incluye pruebas unitarias de dominio y casos de uso, pruebas directas de componentes web/filtros, una suite de repositorios JPA con H2 en Fleet y una referencia separada a la verificación manual de integración con Postman.

No certifica todos los endpoints HTTP, la compatibilidad completa con SQL Server, los contratos entre servicios, el rendimiento, la seguridad integral ni flujos E2E automatizados. Las aplicaciones web y móvil y los repositorios de migraciones de base de datos no forman parte de esta línea base automatizada.

4. COMPONENTES Y ENTORNO DE PRUEBA

Módulo	Java	Spring Boot	Persistencia de producción
rtm-iam	21	4.1.1	JPA / SQL Server
rtm-fleet-maintenance	21	4.1.1	JPA / SQL Server
rtm-booking-reservation	21	4.1.1	JPA / SQL Server
rtm-payment-billing	21	4.1.1	JPA / SQL Server
rtm-api-gateway	21	4.0.8	No administra persistencia de negocio

Las versiones principales de herramientas verificadas son Java 21 y Maven 3.9.16. Los módulos usan JUnit Jupiter mediante `spring-boot-starter-test`; Mockito y AssertJ aparecen en las pruebas existentes. Fleet añade el starter JPA de pruebas y H2 en scope `test`.

Ejecutar con JDK 21, que es la versión declarada en los POM. En este entorno, una ejecución con JDK 27 falló durante la inicialización del procesador de anotaciones Lombok; la línea base se obtuvo con JDK 21.

5. ESTRATEGIA DE PRUEBAS

5.1 Dominio y casos de uso

Probar reglas de negocio con JUnit 5, aislar dependencias externas con dobles cuando corresponda y comprobar resultados, errores e interacciones relevantes. No levantar Spring para pruebas unitarias de lógica que pueda probarse directamente.

5.2 Controladores y filtros

Algunas pruebas invocan métodos de controladores directamente o prueban filtros como unidades aisladas. Sirven para validar delegación y lógica local, pero no atraviesan el servidor MVC ni verifican por sí solas rutas HTTP, serialización JSON, validación de entrada o manejo HTTP integrado.

No se encontraron pruebas `@WebMvcTest`/`MockMvc` en los cinco módulos. Si se necesita validar contratos HTTP, añadir pruebas MVC enfocadas.

5.3 Persistencia

Fleet contiene una suite `@DataJpaTest` con H2 para consultas de sucursales, vehículos, mantenimientos y modelos. H2 valida comportamiento JPA y JPQL, pero no sustituye SQL Server ni cubre todas sus particularidades.

IAM, Booking y Billing usan JPA en producción, pero no se encontraron pruebas `@DataJpaTest`, H2 o `Testcontainers` en sus suites actuales. Sus repositorios, mappings y consultas no se consideran verificados automáticamente por esta línea base.

La suite JPA de Fleet pasa, aunque Hibernate registra un aviso al intentar crear `shared.audit` porque el esquema compartido no está presente en H2. La auditoría y ese esquema quedan fuera de la cobertura de la suite.

5.4 Integración del sistema

La guía de Fleet documenta una verificación manual con Docker, SQL Server, gateway y Postman: 37 de 37 solicitudes exitosas el 4 de octubre de 2026. Es evidencia manual fechada; no forma parte de `mvn test` ni se repitió en la ejecución automatizada del 9 de octubre.

6. PROCEDIMIENTO

6.1 Analizar el cambio

1. Inspeccionar el POM, la estructura de paquetes, las clases fuente y las pruebas cercanas.
2. Identificar el comportamiento existente, los contratos, las dependencias y las excepciones reales.
3. Seleccionar el nivel de prueba que corresponda; no inventar entidades, endpoints ni respuestas.
4. Determinar los límites de lo que la prueba puede demostrar, en especial si utiliza mocks o una base embebida.

6.2 Implementar pruebas

1. Colocar las pruebas bajo `src/test/java`, respetando el paquete del código probado.
2. Aislar casos de uso de Spring y utilizar mocks para dependencias externas cuando sea apropiado.
3. Probar componentes web directamente solo para su lógica local; usar `MockMvc` si se requieren garantías HTTP.
4. Para repositorios JPA, utilizar un slice de persistencia con datos mínimos y documentar la diferencia frente a SQL Server.
5. Cubrir caminos exitosos, alternos y errores relevantes al cambio, con aserciones sobre valores e interacciones.

6.3 Ejecutar desde la raíz de rentacar

Los repositorios son proyectos Maven independientes; no hay un POM agregador común. Ejecutar cada módulo por separado:

```
mvn -f rtm-iam/pom.xml test
mvn -f rtm-fleet-maintenance/pom.xml test
mvn -f rtm-booking-reservation/pom.xml test
mvn -f rtm-payment-billing/pom.xml test
mvn -f rtm-api-gateway/pom.xml test
```

Para ejecutar una suite específica, añadir `-Dtest=NombreDeLaPrueba` antes de `test`. Los reportes de Maven Surefire quedan en `target/surefire-reports` dentro de cada módulo.

6.4 Evaluar y registrar

1. Confirmar compilación y BUILD SUCCESS.
2. Registrar cantidad de pruebas, fallos, errores y omitidas por módulo.
3. Investigar warnings de inicialización aunque no cambien el resultado de la suite.
4. Registrar fecha, Java, comando, rama/commit y alcance de la prueba.
5. Separar evidencia automatizada de pruebas manuales y describir sus límites.

7. INVENTARIO Y RESULTADOS POR MÓDULO

Módulo	Inventario y comportamiento ejercitado	Resultado verificado el 2026-10-09
rtm-iam	Diez clases de prueba, centradas en casos de uso: registro, login/logout, tokens, perfil, correo, contraseña, roles y estado.	34 pruebas; 0 fallos; 0 errores; BUILD SUCCESS
rtm-fleet-maintenance	Casos de uso, controladores por invocación directa, verificación JWT y consultas JPA con H2.	114 pruebas; 0 fallos; 0 errores; 0 omitidas; BUILD SUCCESS
rtm-booking-reservation	Dominio de reservas/rentas/seguros, eventos de pago, expiración y controladores invocados directamente. Dos repositorios en memoria son auxiliares de prueba, no suites.	53 pruebas; 0 fallos; 0 errores; BUILD SUCCESS
rtm-payment-billing	Dos clases de prueba: entidad Payment y caso de uso PaymentUseCaseImpl.	14 pruebas; 0 fallos; 0 errores; BUILD SUCCESS
rtm-api-gateway	Tres clases de prueba: filtro JWT, filtro de correlation ID y limitador en memoria.	10 pruebas; 0 fallos; 0 errores; BUILD SUCCESS
Total backend Java	Ejecuciones independientes por módulo	225 pruebas; 0 fallos; 0 errores

La suma de 225 es un agregado de cinco ejecuciones Maven independientes, no una ejecución de reactor multi-módulo.

8. CRITERIOS DE ACEPTACIÓN

- El módulo afectado compila con Java 21 y su suite completa termina sin fallos ni errores.
- Los nombres y escenarios corresponden a clases y comportamiento existentes.
- Las pruebas son repetibles y no dependen de servicios remotos o credenciales reales, salvo en procedimientos manuales controlados.
- Una prueba de slice declara el motor de base de datos y las diferencias respecto a producción.
- Las comprobaciones manuales no se presentan como automatizadas.
- No se informa porcentaje de cobertura si no existe una medición configurada.

9. BRECHAS Y RIESGOS CONOCIDOS

Prioridad	Brecha	Alcance afectado
Alta	Falta validación HTTP real con @WebMvcTest/MockMvc; pruebas directas no prueban transporte ni serialización.	Los cinco módulos; especialmente controllers de Fleet y Booking
Alta	No hay pruebas de repositorio en tres módulos JPA; Fleet usa H2, no SQL Server, y no prueba shared.audit.	IAM, Booking, Billing y límites de Fleet
Alta	Billing tiene dos clases de prueba y 14 casos, el inventario más pequeño por servicio.	Billing
Media	No hay pruebas automáticas de rutas/upstreams del Gateway en contexto HTTP.	API Gateway
Media	JaCoCo no está configurado, así que no existe porcentaje de cobertura reproducible.	Todos
Media	No hay pipeline CI configurado para ejecutar las suites al integrar cambios.	Todos
Media	La verificación integrada de Postman es manual, no repetible mediante mvn test.	Sistema completo

10. ACCIONES DE MEJORA RECOMENDADAS

1. Priorizar pruebas de reglas de negocio y errores de pago en Billing.
2. Añadir pruebas MVC con MockMvc para endpoints y manejo de errores HTTP.
3. Añadir pruebas de persistencia para IAM, Booking y Billing; evaluar Testcontainers con SQL Server para compatibilidad real.
4. Completar pruebas del enrutamiento y upstreams del Gateway.
5. Configurar JaCoCo con metas acordadas y agregar mvn test a CI por repositorio.
6. Automatizar gradualmente los flujos Postman/E2E más críticos.

Estas acciones son propuestas; no forman parte de los resultados ejecutados aquí.

11. ROLES Y RESPONSABILIDADES

Rol	Responsabilidad
Desarrollador	Implementar pruebas del cambio, ejecutarlas en su módulo y registrar los resultados.
Revisor técnico	Confirmar que los escenarios corresponden al código real y que se declaran sus límites.
Responsable de calidad o líder	Aprobar criterios de aceptación y priorizar las brechas.

Rol	Responsabilidad
Responsable del entorno	Mantener los servicios y datos de prueba requeridos para validaciones manuales o integradas.